You are a senior hackathon engineering team consisting of:

* Senior full-stack engineer
* Bioprocess engineer
* ML/data scientist
* Scientific-computing engineer
* UI/UX engineer
* DevOps engineer
* Hackathon product/demo expert

Build a competition-ready MVP called:

BioCopilot

AI Copilot for Scalable Cell-Culture Bioprocess Design

Target event:
YUVA Megathon 2026 — SRM IST Trichy — Biotech Track

Core positioning:

"From cell-culture data to explainable process decisions — detect metabolic risk, predict process behavior, optimize nutrients and cost, and evaluate scale-up from 2 L toward 1,000 L."

The application is a DECISION-SUPPORT SYSTEM.

It must NOT directly control laboratory equipment.

Every recommendation must require human review/approval.

---

1. PRIMARY OBJECTIVE

Build a real, functional web application — NOT a static UI mockup.

The complete workflow must be:

DATA
↓
DATA VALIDATION
↓
FEATURE ENGINEERING
↓
ML / DIGITAL-TWIN ANALYSIS
↓

REAL-TIME SIMULATED SENSOR MONITORING
↓
RISK DETECTION
↓
PREDICTION
↓
NUTRIENT / FEED OPTIMIZATION
↓
MEDIA COST ANALYSIS
↓
2 L → 1,000 L SCALE-UP
↓
EXPLAINABLE RECOMMENDATION
↓
HUMAN APPROVAL
↓
REPORT

Prioritize a smaller number of deeply functional features over many fake features.

---

2. HACKATHON PRIORITY

The judging experience is more important than feature count.

Prioritize:

P0 — MUST WORK

1. Dataset ingestion
2. Data-quality validation
3. Interactive process dashboard
4. Simulated live sensors
5. Lactate/metabolic risk detection
6. VCD/viability prediction
7. Explainable AI
8. Feed/nutrient what-if optimization
9. Media-cost comparison
10. 2 L → 1,000 L scale-up
11. Human approval
12. Demo mode

P1 — SHOULD WORK

13. Historical batch comparison

14. Anomaly detection
15. Report generation
16. Model validation page
17. AI Copilot chat

P2 — NICE TO HAVE

18. Advanced Bayesian optimization
19. Nutritional/PDCAAS-style analysis
20. Optional external LLM
21. Advanced sensor fault injection

If time becomes limited, NEVER sacrifice P0 stability for P2 features.

---

3. IMPLEMENTATION-FIRST WORKFLOW

FIRST create:

/docs/implementation_plan.md

The plan must contain:

* Architecture
* Technology stack
* Folder structure
* Data pipeline
* ML pipeline
* Sensor architecture
* API contract
* Database design
* Frontend pages
* Demo workflow
* Testing strategy
* Risks
* Fallback strategy

Then create:

/docs/api_contract.md

Before implementing the modules, make every agent follow this API contract.

---

4. PARALLEL DEVELOPMENT AGENTS

Create these logical workstreams:

AGENT A — DATA + SENSOR ENGINE

Responsible for:

* Dataset ingestion
* Dataset adapter
* Canonical schema
* Data cleaning
* Data-quality report
* Dataset provenance
* Virtual sensors
* Sensor replay
* Fault injection

AGENT B — ML + OPTIMIZATION

Responsible for:

* Feature engineering
* Model training
* VCD prediction
* Viability prediction
* Lactate prediction
* Ammonia prediction
* Metrics
* SHAP/feature importance
* Nutrient analysis
* Feed optimization
* Cost optimization

AGENT C — SCALE-UP ENGINE

Responsible for:

* 2 L
* 10 L
* 100 L
* 1,000 L

Calculate scenario comparisons for:

* Constant P/V
* Constant tip speed
* Constant kLa

* Constant mixing time

Clearly label engineering calculations as estimates.

AGENT D — FASTAPI BACKEND

Responsible for:

* REST APIs
* WebSocket/SSE sensor stream
* Pydantic schemas
* SQLite database
* Audit log
* Recommendation approval
* Report generation

AGENT E — REACT FRONTEND

Responsible for:

* Dashboard
* Charts
* Sensor monitor
* Prediction UI
* Optimization UI
* Scale-up UI
* Risk center
* Explainability
* Human approval
* Demo mode

---

5. DATA STRATEGY

Use verified public cell-culture/bioreactor data where available.

Search/configure an appropriate Kaggle dataset containing as many as possible of:

* VCD
* viability
* glucose
* glutamine
* lactate
* ammonia
* pH

* DO
* temperature
* agitation
* gas flow
* feed rate
* batch/time information

DO NOT invent:

* Dataset names
* Kaggle URLs
* Experimental measurements
* Sensor values
* Licenses
* Scientific results

Create:

/data/raw
/data/processed
/data/reference
/data/metadata

Create:

/config/data_source.yaml

containing:

dataset_name:
dataset_identifier:
dataset_url:
license:
download_date:
selected_file:
column_mapping:

If internet/Kaggle access is unavailable:

Use a clearly labelled synthetic/demo dataset.

The UI MUST display:

"SYNTHETIC DEMO DATA — NOT EXPERIMENTAL DATA"

Never present synthetic data as experimental data.

---

6. CANONICAL DATA SCHEMA

Normalize source datasets into:

time_h
batch_id
vessel_volume_L
VCD
viability_pct
glucose_gL
glutamine_mM
lactate_gL
ammonia_mM
pH
DO_pct
temp_C
rpm
gas_flow_vvm
feed_rate_mL_h
media_cost_per_L

Not every dataset will contain every variable.

If a variable is unavailable:

display:

"Not available in source data"

Never fabricate it.

---

7. DATA ADAPTER

Implement automatic column mapping.

Example:

glucose_mM → glucose_gL
viability → viability_pct
DO_percent → DO_pct
cell_density → VCD

Use fuzzy matching only as a suggestion.

The user must be able to manually confirm/correct the mapping.

Display:

* source column
* canonical variable
* unit
* datatype
* missing %
* validity
* model usage

---

8. DATA QUALITY ENGINE

Implement:

* duplicate detection
* missing-value analysis
* interpolation where scientifically appropriate
* outlier detection
* physiological range checks
* datatype conversion
* unit normalization
* timestamp normalization
* batch identification
* feature correlation
* data-quality flags

Physiological checks should include configurable ranges such as:

pH: 6.0–8.0
viability: 0–100%

Do not silently modify raw data.

Create:

/data/processed/

and:

/docs/data_quality_report.md

Show a Data Quality Score using transparent rules.

---

9. FEATURE ENGINEERING

Calculate when sufficient data exists:

* specific growth rate μ
* glucose consumption rate
* glutamine consumption rate
* lactate production rate
* ammonia production rate
* lactate/glucose relationship
* IVCD
* rolling trends
* slopes
* moving averages
* process deviation features

Never calculate a feature when required input data is unavailable.

---

10. MACHINE LEARNING ENGINE

Primary targets:

1. Future VCD
2. Future viability
3. Future lactate
4. Future ammonia

Candidate models:

* Linear Regression baseline
* Random Forest
* Gradient Boosting
* XGBoost if available

Do not blindly select a model.

Compare models.

Display:

MODEL
MAE
RMSE
R^2
TRAINING SAMPLES
VALIDATION SAMPLES
TEST SAMPLES

CRITICAL:

Split by batch/run.

Never randomly split individual time-series rows when this causes leakage.

If data is insufficient:

Do NOT fake metrics.

Display:

"Insufficient data for reliable predictive modelling."

Use analytical/rule-based decision support instead.

---

11. PREDICTION UNCERTAINTY

Where feasible implement:

* bootstrap intervals
  OR
* quantile regression
  OR
* model ensemble uncertainty

Display:

Prediction
Lower bound
Upper bound
Confidence

Never present prediction as guaranteed biological outcome.

---

12. EXPLAINABLE AI

For each important prediction provide:

WHY?

Show:

* top contributing features
* feature direction
* model contribution
* confidence
* constraints
* data source

Use SHAP when technically feasible.

Otherwise use model feature importance.

Clearly distinguish:

MODEL CONTRIBUTION

from:

BIOLOGICAL CAUSATION

Example:

"Recent lactate increase is strongly associated with the model's predicted viability change."

Do NOT state:

"Lactate caused the viability decrease"

unless experimentally established.

---

13. METABOLIC RISK ENGINE

Detect:

* glucose depletion
* excessive glucose

* rising lactate
* rising ammonia
* glutamine depletion
* abnormal VCD
* falling viability
* DO limitation
* pH instability
* abnormal process trajectory
* sensor anomalies

Each alert must contain:

severity
parameter
observed value
trend
evidence
possible driver
predicted impact
recommended review
confidence

Severity:

INFO
LOW
MEDIUM
HIGH
CRITICAL

---

14. VIRTUAL SENSOR ENGINE

Implement simulated sensors:

SENSOR-01 pH
SENSOR-02 DO
SENSOR-03 Temperature
SENSOR-04 Glucose
SENSOR-05 Glutamine
SENSOR-06 Lactate
SENSOR-07 Ammonia
SENSOR-08 VCD
SENSOR-09 Viability
SENSOR-10 RPM
SENSOR-11 Gas Flow

SENSOR-12 Feed Rate

Stream every 2 seconds.

Use:

WebSocket:

/ws/sensors

Support:

* historical replay
* synthetic replay
* current value
* previous value
* trend
* rolling average
* threshold
* warning
* critical

Display prominently:

SIMULATED SENSOR DATA

Never imply connection to physical equipment.

---

15. FAULT INJECTION

Create demo buttons:

"Inject Lactate Rise"

"Inject Glucose Overfeed"

"Inject DO Sensor Failure"

"Inject pH Drift"

"Inject Viability Crash"

The fault must modify the simulated stream only.

It must trigger the risk engine.

This is one of the PRIMARY judge-demo features.

---

16. NUTRIENT ANALYSIS

Compare current process values against:

* historical good batches
* configurable reference ranges
* user-defined targets

Classify:

DEFICIENT
OPTIMAL
EXCESS
CRITICAL
UNKNOWN

Use gauges/heatmaps.

Never claim a reference range is universally valid.

Show:

"Reference range depends on process, cell line and medium formulation."

---

17. SMART FEED SCHEDULER

Inputs:

* current glucose
* glutamine
* VCD
* viability
* lactate
* ammonia
* pH
* DO
* feed rate
* process time

Generate bounded decision-support scenarios.

Example:

CURRENT PLAN

versus

SIMULATED PLAN

Display predicted:

VCD
viability
glucose
lactate
ammonia
cost

Every recommendation must include:

WHY
EXPECTED EFFECT
RISK
CONFIDENCE
CONSTRAINTS CHECKED
DATA SOURCE

And:

"HUMAN APPROVAL REQUIRED"

Never send equipment-control commands.

---

18. MEDIA COST OPTIMIZER

Allow editable price table.

Currencies:

INR
USD

Components can include:

* base media

* glucose
* glutamine
* growth factors
* supplements
* hydrolysates
* recombinant alternatives

Do not invent prices.

If price is unavailable:

allow manual entry.

Calculate:

Current cost/batch
Optimized scenario cost
Cost/L
Cost per 10^9 cells
Estimated savings

Savings must be calculated from the supplied values.

Never hard-code:

"70% cost reduction"

or any other unsupported percentage.

---

19. PARETO OPTIMIZATION

Where enough data exists, implement bounded optimization.

Objective:

MINIMIZE COST

while attempting to maintain:

viability >= configurable threshold
lactate <= configurable threshold
ammonia <= configurable threshold
nutrient adequacy

Show:

COST vs GROWTH

and:

COST vs VIABILITY

Clearly label simulated optimization as:

MODEL ESTIMATE

---

20. SCALE-UP ENGINE

Build:

2 L → 10 L → 100 L → 1,000 L

Strategies:

1. Constant P/V
2. Constant tip speed
3. Constant kLa
4. Constant mixing time

Use documented engineering correlations where inputs are available.

For example:

$P = Np \times \rho \times N^3 \times D^5$

$kLa = k \times (P/V)^a \times (vvs)^b$

Use configurable defaults:

a = 0.4
b = 0.5

Calculate when sufficient input data exists:

* power/volume
* tip speed
* Reynolds number
* estimated mixing time
* gas velocity
* kLa

* oxygen transfer scenario
* DO risk
* shear-risk indicator

If required engineering parameters are unavailable:

DO NOT fabricate them.

Display:

"Insufficient process data for this calculation."

Create a comparison table:

Strategy
RPM
Gas flow
P/V
Tip speed
kLa estimate
Mixing estimate
DO risk
Shear risk

Mark all outputs:

"ENGINEERING ESTIMATE — REQUIRES VALIDATION"

---

21. DIGITAL-TWIN APPROXIMATION

Create a lightweight mechanistic simulator.

It may use simplified:

* Monod growth
* substrate limitation
* lactate inhibition
* ammonia inhibition
* cell death
* oxygen-transfer limitation

This is ONLY a digital-twin approximation.

The UI must say:

"DIGITAL TWIN APPROXIMATION — NOT GROUND TRUTH"

Do not claim the simulator represents a validated biological model.

---

22. WHAT-IF SIMULATOR

Allow users to change:

Glucose feed
Glutamine feed
Feed timing
Gas flow
RPM
DO target

Show:

CURRENT

vs

SIMULATED

For:

VCD
Viability
Glucose
Lactate
Ammonia
Cost

The simulation is decision support only.

---

23. HUMAN-IN-THE-LOOP

Every AI recommendation must follow:

TEST
↓
PENDING
↓

APPROVE / REJECT
↓
COMMENT
↓
AUDIT LOG

Store:

timestamp
user/action
dataset version
model version
recommendation
input state
decision
comment

Nothing is automatically applied.

---

24. AI COPILOT

Create a chat-style assistant.

Supported questions:

"Why is lactate increasing?"

"What is affecting viability?"

"What are the current process risks?"

"Compare this batch with historical batches."

"What changes from 2 L to 200 L?"

"Why is this batch different?"

"Explain this prediction."

"Which feed scenario has lower estimated cost?"

The assistant MUST answer using application state.

Default implementation:

LOCAL TEMPLATE-BASED COPILOT

No paid API.

Optional LLM provider can be added behind an environment variable, but it must be OFF by default.

If information is unavailable:

"Insufficient data available to answer this reliably."

Never hallucinate experimental results.

---

25. DASHBOARD DESIGN

Create a professional biotechnology/industrial AI interface.

Dark "laboratory instrument" theme.

Include light mode.

Responsive.

Navigation:

Overview
Data
Live Monitor
AI Predictions
Metabolic Analysis
Nutrient Optimizer
Feed Scheduler
Media Cost
Scale-Up
Risk Center
Explainability
Reports

Use:

* Plotly charts
* KPI cards
* process-health indicators
* alert timeline

* trend lines
* badges
* status indicators
* comparison tables

Avoid meaningless AI animations.

Every visual must communicate useful process information.

---

26. MOST IMPORTANT HOME SCREEN

The home page must immediately communicate:

THE PROBLEM

High media cost
Metabolic imbalance
Process variability
Scale-up uncertainty

↓

THE SOLUTION

BioCopilot

↓

THE PIPELINE

DATA
→
DETECT
→
PREDICT
→
OPTIMIZE
→
SCALE
→
APPROVE

Show a compact live process summary.

---

27. DEMO MODE

Create a dedicated:

"START 5-MIN DEMO"

button.

The demo must automatically guide the judge through:

STEP 1
Load verified dataset/demo dataset.

STEP 2
Show data quality.

STEP 3
Select a batch.

STEP 4
Show healthy process.

STEP 5
Start simulated sensors.

STEP 6
Inject "Glucose Overfeed".

STEP 7
Show lactate rising.

STEP 8
Risk engine detects the issue.

STEP 9
Prediction shows potential impact.

STEP 10
Explainability shows contributing signals.

STEP 11
Feed optimizer generates two bounded scenarios.

STEP 12
Cost comparison appears.

STEP 13
Scale-up 2 L → 1,000 L shows engineering trade-offs.

STEP 14
Recommendation enters human approval.

STEP 15
Approve/reject with comment.

STEP 16
Generate report.

The demo must be deterministic.

---

28. THREE-MINUTE JUDGE STORY

The primary narrative should be:

"Here is a real cell-culture process."

"The data quality is checked before AI is allowed to make predictions."

"Now we replay the process as simulated sensors."

"We inject an overfeeding event."

"The system detects rising lactate."

"Instead of simply showing an alert, BioCopilot explains the signals associated with the prediction."

"Now we test alternative feed scenarios."

"We compare expected growth, viability, metabolic risk and estimated cost."

"Finally, we ask what happens when this process moves from 2 L toward 1,000 L."

"The scale-up engine shows where oxygen transfer, mixing and shear assumptions become important."

"Nothing is automatically applied."

"A human must approve the recommendation."

This should be the core demo.

---

29. API CONTRACT

Implement:

GET /health

POST /api/datasets/upload
GET /api/datasets
POST /api/datasets/{id}/clean

GET /api/runs/{id}

GET /ws/sensors
POST /api/sensors/fault

POST /api/analysis/nutrients
POST /api/predict/metabolites

POST /api/optimize/media
POST /api/feed/schedule

POST /api/scaleup

GET /api/alerts

POST /api/recommendations/{id}/approve
POST /api/recommendations/{id}/reject

GET /api/audit

GET /api/report/{run_id}

Also provide:

GET /api/model/metrics
POST /api/copilot/query

Use Pydantic validation.

---

30. PROJECT STRUCTURE

Create:

biocopilot/

backend/
app/
main.py
api/
models/
schemas/
services/
simulators/
ml/
database/
utils/
tests/

frontend/
src/
pages/
components/
charts/
api/
hooks/
types/

data/
raw/
processed/
reference/
demo/

notebooks/

docs/
implementation_plan.md
architecture.md
api_contract.md
demo_script.md
limitations.md
data_sources.md

config/
data_source.yaml

Makefile

docker-compose.yml
Dockerfile
README.md
.env.example

---

31. TECHNOLOGY

Backend:

Python 3.11
FastAPI
Pydantic
SQLAlchemy
SQLite
Pandas
NumPy
SciPy
scikit-learn
XGBoost where available
Optuna where useful
SHAP where feasible

Frontend:

React
TypeScript
Vite
Tailwind CSS
Plotly
React Router

Deployment-ready:

Docker
Vercel-compatible frontend
Render/Railway-compatible backend

No paid APIs.

No required secrets.

The application must run locally.

---

32. ONE-COMMAND STARTUP

Implement:

make dev

or:

docker compose up

The application must start with one command.

Preload a demo dataset so the dashboard is populated immediately.

---

33. TESTING

Write tests for:

* dataset loading
* column mapping
* cleaning
* missing values
* outliers
* feature engineering
* batch splitting
* model training
* model metrics
* risk detection
* recommendation constraints
* scale-up mathematics
* API validation
* virtual sensor stream
* recommendation approval

Run all tests.

Show the final result.

Do not claim tests pass unless they actually pass.

---

34. VISUAL QA

After each major milestone:

1. Start the application.
2. Open it in the browser.
3. Visit every major page.
4. Check console/runtime errors.
5. Check API failures.
6. Check chart rendering.
7. Check responsive layout.
8. Fix visual bugs.
9. Fix runtime bugs.
10. Capture screenshots.

Create:

/docs/walkthrough.md

Include:

* screenshot
* page name
* purpose
* working features
* known limitations

---

35. LIMITATIONS PANEL

Always show:

LIMITATIONS

* Model performance depends on dataset size and quality.
* Synthetic data is not experimental data.
* Scale-up equations are engineering approximations.
* Digital twin is not ground truth.
* Model correlation does not prove biological causation.
* Cost estimates depend on supplied prices.
* Recommendations require qualified human review.
* Wet-lab validation is required before real implementation.
* The prototype does not control laboratory equipment.

---

36. README

Create a professional README with:

Project Overview
Problem
Solution
Architecture
Data Pipeline
Dataset Sources
Dataset Licenses
ML Pipeline
Feature Engineering
Virtual Sensors
Risk Engine
Optimization
Scale-Up
Explainable AI
Human Validation
API
Frontend
Backend
Testing
Demo
Deployment
Limitations
Future Work

Include Mermaid architecture diagram.

---

37. FINAL QUALITY GATE

Before declaring completion, verify:

[ ] Frontend starts

[ ] Backend starts

[ ] Database starts

[ ] Dataset loads

[ ] Dataset provenance appears

[ ] Data quality report works

[ ] Charts work

[ ] Sensor stream works

[ ] Fault injection works

[ ] Lactate risk detection works

[ ] Prediction works when sufficient data exists

[ ] Metrics are real

[ ] Explainability works

[ ] Feed scenario works

[ ] Cost calculation works

[ ] Scale-up calculation works

[ ] Human approval works

[ ] Audit log works

[ ] Report generation works

[ ] Tests pass

[ ] No fake dataset claims

[ ] No fabricated experimental results

[ ] Synthetic data is clearly labelled

[ ] No major console errors

[ ] No major API errors

[ ] Demo mode works from a clean startup

---

38. FINAL PRODUCT EXPERIENCE

The judge should understand the entire product within 30 seconds.

The first impression should be:

"BioCopilot watches the process, understands what is happening, predicts what may happen next, tests safer/lower-cost scenarios, evaluates scale-up risk, explains why, and asks a human before any recommendation is accepted."

Do NOT optimize for maximum number of pages.

Optimize for:

SCIENTIFIC CREDIBILITY
+
TECHNICAL EXECUTION
+
VISIBLE AI FUNCTIONALITY
+
EXPLAINABILITY
+
DEMOABILITY
+
HONESTY

Build the actual working application now.

Do not stop at UI mockups.

Do not create fake AI outputs.

When a capability cannot be supported by the available data, explicitly show:

"Insufficient data."

When using synthetic data, explicitly show:

"SYNTHETIC DEMO DATA — NOT EXPERIMENTAL DATA."

When using engineering approximations, explicitly show:

"ENGINEERING ESTIMATE — REQUIRES VALIDATION."

Every recommendation must show:

WHY
EVIDENCE
EXPECTED EFFECT
CONFIDENCE

CONSTRAINTS
DATA SOURCE
HUMAN APPROVAL REQUIRED

Final positioning:

"BioCopilot transforms cell-culture data into explainable process intelligence — helping teams detect metabolic risk, predict process behavior, evaluate nutrient/feed scenarios, estimate economics, and assess scale-up from 2 L toward 1,000 L."